

Comparação Entre Algoritmos de Escalonamento

Breno C. R. Nakamura¹, Estevan T. Santos¹, Gabriel L. Bonavina¹,
Jonas L. Durão¹, Natália V. A. do Val¹, Vinicius A. L. Andrade¹

¹Instituto de Ciência e Tecnologia – Universidade Federal de São Paulo (UNIFESP)
Av. Cesare Monsueto Giulio Lattes, 1201 – 12247-014 – São José dos Campos, SP

Abstract. *This meta-paper describes the style to be used in articles and short papers for SBC conferences. For papers in English, you should add just an abstract while for the papers in Portuguese, we also ask for an abstract in Portuguese (“resumo”). In both cases, abstracts should not have more than 10 lines and must be in the first page of the paper.*

Resumo. *Este meta-artigo descreve o estilo a ser usado na confecção de artigos e resumos de artigos para publicação nos anais das conferências organizadas pela SBC. É solicitada a escrita de resumo e abstract apenas para os artigos escritos em português. Artigos em inglês deverão apresentar apenas abstract. Nos dois casos, o autor deve tomar cuidado para que o resumo (e o abstract) não ultrapassem 10 linhas cada, sendo que ambos devem estar na primeira página do artigo.*

1. Introdução

2. Instalação

A primeira etapa deste projeto consistiu na instalação e configuração inicial do MINIX na máquina virtual. Para isso, foi necessário seguir o manual de instruções apresentado no enunciado. Imagens da instalação podem ser encontradas nas Figuras 1 e 2.

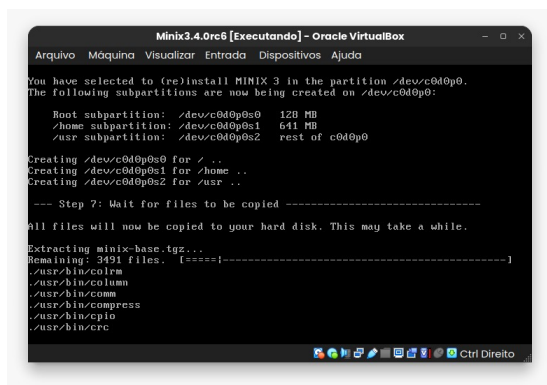


Figura 1. Etapa 7 de instalação do MINIX.

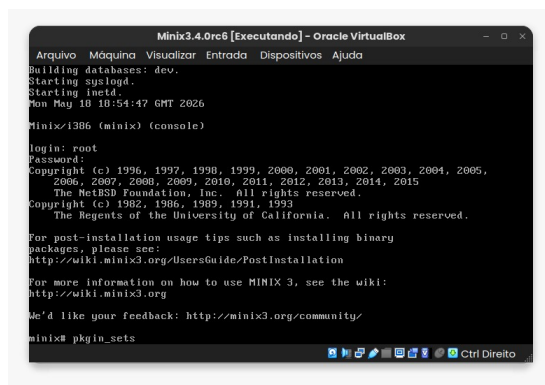


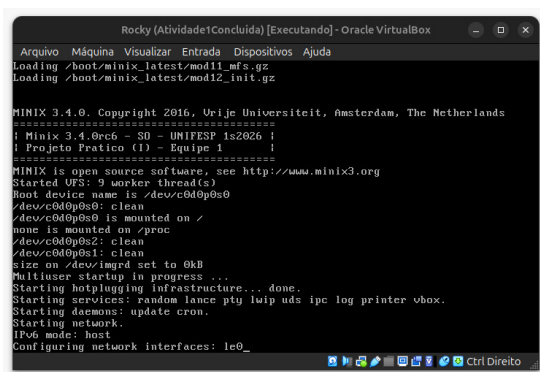
Figura 2. Atualização dos pacotes do MINIX.

Além da instalação do MINIX, também foi necessário fazer a configuração de rede no *Virtual Box*, ajustando a política de endereçamento MAC para incluir apenas os endereços MAC de placas em rede NAT.

Além disso, foi necessário fazer o *clone* do repositório no ambiente virtual, para que possamos fazer as modificações que serão detalhadas em seções futuras deste relatório. Para isso, foi necessário configurar um Token de Acesso Pessoal do Github, o que nos permitiu executar os comandos do *git* no ambiente virtual.

3. Mudança nos Banners

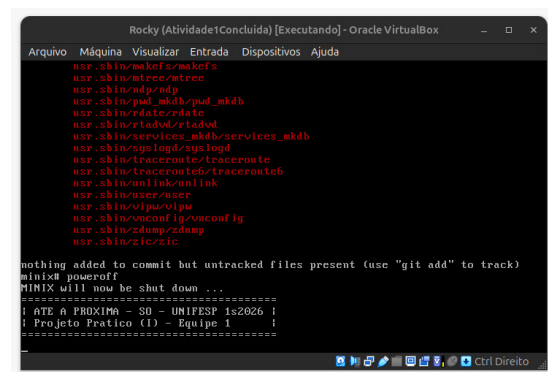
Para a mudança no banner durante *boot* e *poweroff*, foi necessário navegar no diretório `minix/minix/kernel/main.c`. Na função `announce()`, foram acrescentados *prints* para reproduzir o banner indicado no enunciado. Além disso, na função `prepare_shutdown()`, também inserimos *prints* para a impressão do banner. Os resultados podem ser encontrados nas Figuras 3 e 4.



```
Rocky (Atividade1Concluida) [Executando] - Oracle VirtualBox
Arquivo  Máquina  Visualizar  Entrada  Dispositivos  Ajuda
Loading /boot/minix_latest/mod11_mfs.gz
Loading /boot/minix_latest/mod12_init.gz

MINIX 3.4.0. Copyright 2016, Urije Universiteit, Amsterdam, The Netherlands
=====
! Minix 3.4.0rc6 - SO - UNIFESP 1s2026 !
! Projeto Pratico (I) - Equipe 1 !
=====
MINIX is open source software, see http://www.minix3.org
Started UFS: 9 worker thread(s)
Boot device name is /dev/c0d0p0s0
/dev/c0d0p0s0: clean
/dev/c0d0p0s0 is mounted on /
none is mounted on /proc
/dev/c0d0p0s2: clean
/dev/c0d0p0s1: clean
size on /dev/lmgrd set to 0kH
Multiuser startup in progress ...
Starting hotplugging infrastructure... done.
Starting services: random lance pty lwp uds ipc log printer vbox.
Starting daemons: update cron.
Starting network.
IPv6 mode: host
Configuring network interfaces: le0_
```

Figura 3. Resultado do banner de *boot*.

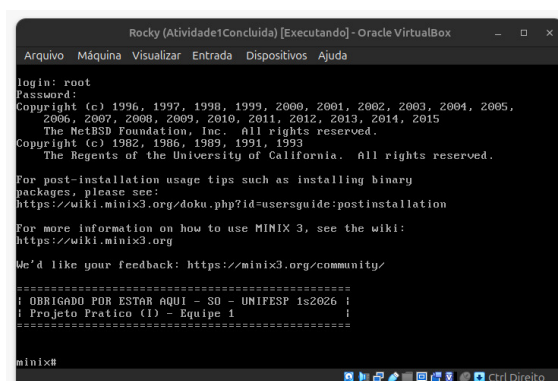


```
Rocky (Atividade1Concluida) [Executando] - Oracle VirtualBox
Arquivo  Máquina  Visualizar  Entrada  Dispositivos  Ajuda
usr/sbin/makefs/makefs
usr/sbin/mtree/mtree
usr/sbin/ndp/ndp
usr/sbin/pad_mkdb/pad_mkdb
usr/sbin/rdate/rdate
usr/sbin/rtdad/rtdad
usr/sbin/services_mkdb/services_mkdb
usr/sbin/swaplogd/swaplogd
usr/sbin/traceroute/traceroute
usr/sbin/traceroute6/traceroute6
usr/sbin/unlink/unlink
usr/sbin/over/over
usr/sbin/vlpa/vlpa
usr/sbin/vnconfig/vnconfig
usr/sbin/zdump/zdump
usr/sbin/zic/zic

nothing added to commit but untracked files present (use "git add" to track)
minix poweroff
MINIX will now be shut down ...
=====
! ATE A PROXIMA - SO - UNIFESP 1s2026 !
! Projeto Pratico (I) - Equipe 1 !
=====
```

Figura 4. Resultado do banner de *poweroff*.

Por fim, para a modificação do banner de *login*, foi necessário navegar pelos diretórios `minix/etc/motd`, onde inserimos o banner indicado pelo enunciado. O resultado da modificação pode ser encontrado na Figura 5



```
Rocky (Atividade1Concluida) [Executando] - Oracle VirtualBox
Arquivo  Máquina  Visualizar  Entrada  Dispositivos  Ajuda

login: root
Password:
Copyright (c) 1996, 1997, 1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005,
2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015
The NetBSD Foundation, Inc. All rights reserved.
Copyright (c) 1982, 1986, 1989, 1991, 1993
The Regents of the University of California. All rights reserved.

For post-installation usage tips such as installing binary
packages, please see:
https://wiki.minix3.org/doku.php?id=usersguide:postinstallation

For more information on how to use MINIX 3, see the wiki:
https://wiki.minix3.org

We'd like your feedback: https://minix3.org/community/

=====
! OBRIGADO POR ESTAR AQUI - SO - UNIFESP 1s2026 !
! Projeto Pratico (I) - Equipe 1 !
=====

minix#
```

Figura 5. Resultado do banner de *login*.

4. Mudança `exec.c`

Para registrar a execução de processos exibindo o comando e seu caminho absoluto, a implementação foi dividida entre o *Virtual File System* (VFS) e o *Process Manager* (PM), respeitando a modularidade da arquitetura do MINIX [MINIX 3 Project].

Inicialmente, adicionou-se o campo `execpath` à estrutura compartilhada `exec_info` (no arquivo `libexec.h`) como um campo de comunicação entre o VFS e o PM. No VFS (`vfs/exec.c`), logo após o carregamento do executável, o caminho absoluto armazenado na variável `finalexec` é copiado para `execi.args.execpath` utilizando a função `strncpy`. Em seguida, o VFS aciona o PM por meio de `libexec_pm_newexec`.

A exibição da mensagem foi centralizada no PM (`pm/exec.c`). Na rotina `do_newexec`, os dados preenchidos pelo VFS são recebidos via `sys_datacopy`. Após a cópia dos argumentos para a memória do PM, inseriu-se um `printf` para exibir a mensagem Executando: `<caminho/comando>`. O resultado dessa modificação pode ser encontrado na Figura 6.

Figura 6. Resultado da modificação do `exec.c`

5. Descrição do Exercício Proposto

Este projeto tem como objetivo avaliar o desempenho de diferentes algoritmos de escalonamento no sistema operacional MINIX 3.4.0rc6. A metodologia consiste na análise teórica e na modificação do código-fonte do sistema para implementar três políticas de escalonamento distintas, além da abordagem padrão do MINIX. Para isso, foram implementados os algoritmos: Round-Robin, Múltiplas Filas e *First-Come, First-Served* (FCFS).

Para analisar o comportamento e a eficiência de cada algoritmo sob diferentes perfis de carga, serão conduzidos testes comparativos simulando cenários com processos intensivos em computação (*CPU-bound*) e intensivos em entrada/saída (*I/O-bound*). A avaliação quantitativa será realizada por meio do programa de testes `teste_processos.c`, variando-se a carga de trabalho em escalas de 10, 50, 100 e 200 processos simultâneos. Adicionalmente, o sistema será avaliado em cenários extremos, submetendo os escalonadores à execução de 10 mil processos *I/O-bound* e 1 milhão de processos *CPU-bound*.

6. Descrição dos Algoritmos de Escalonamento

6.1. Algoritmo Padrão do MINIX

6.2. Round-Robin

O algoritmo de escalonamento por chaveamento circular (ou *Round-Robin*), é um dos algoritmos mais simples e antigos devido à sua fácil implementação. Seu funcionamento gira em torno da definição de uma fatia de tempo de CPU (*quantum*), em que o processo que está na CPU terá um tempo limitado de uso e, ao final deste tempo, haverá preempção e um novo processo será escolhido para ser executado [Tanenbaum 2016].

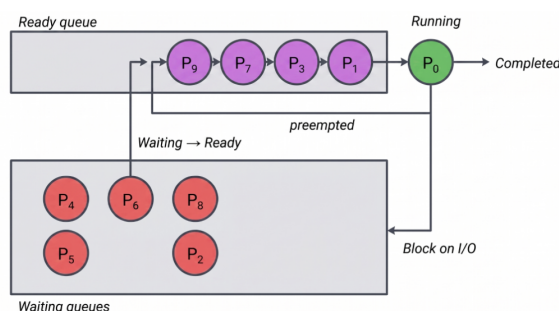


Figura 7. Diagrama de funcionamento do *Round-Robin* [da Costa 2026]

Para implementá-lo, é necessária a utilização de uma fila circular de processos prontos. Para nossa implementação, substituímos as múltiplas filas do algoritmo padrão do MINIX por uma fila única de processos de usuário (USER_Q 7), além disso, desabilitamos os algoritmos de demção, promoções periódicas de prioridade e rebalanceamento de filas. Por fim, modificamos o *quantum* para que ele seja variável, com a seguinte lógica: cada processo terá uma fatia de CPU inversamente proporcional ao número de processos de usuário prontos. Em sua fila circular, quando o *quantum* do processo em execução acabar, ele deverá ser encaminhado para o final da fila. Do mesmo modo, quando ele for bloqueado por entrada e saída, quando seu estado for atualizado para pronto, ele deverá retornar ao final da fila, como pode ser visto na Figura 7.

6.3. Múltiplas Filas

6.4. Algoritmo a ser escolhido

7. Resultados Obtidos e Discussão

Para garantir a robustez estatística de nossos resultados, realizamos 5 execuções para cada caso, e calculamos a média e o desvio padrão de cada algoritmo. Ao final das execuções, foi confeccionado um gráfico de barras para cada algoritmo, onde o eixo x representa o número de processos e o eixo y representa o tempo de execução médio.

Referências

da Costa, J. B. D. (2026). Capítulo 2 - processos e threads. Slides apresentados na disciplina de Sistemas Operacionais.

MINIX 3 Project. Minix 3 wiki documentation. <https://wiki.minix3.org/>. Accessed: 2026-06-04.

Tanenbaum, A. S. (2016). *Sistemas Operacionais Modernos*. São Paulo: Pearson Education Brasil, 4th edition.